

Teil 4:

Test und Verifikation

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Lernziele

- Repräsentationen eines Programmes, auf denen Verifikation stattfinden kann, **benennen** und **verstehen**



- Metriken auf Basis der CFG **ausrechnen**



- Spezifikationen in strukturierter operationeller Semantik **verstehen** und **erläutern**



- Beweise für mögliche Ausführung von While Programmen mit Hilfe von S.O.S. Regeln **durchführen**



- Symbolische Ausführung für While Programme **durchführen**



Agenda

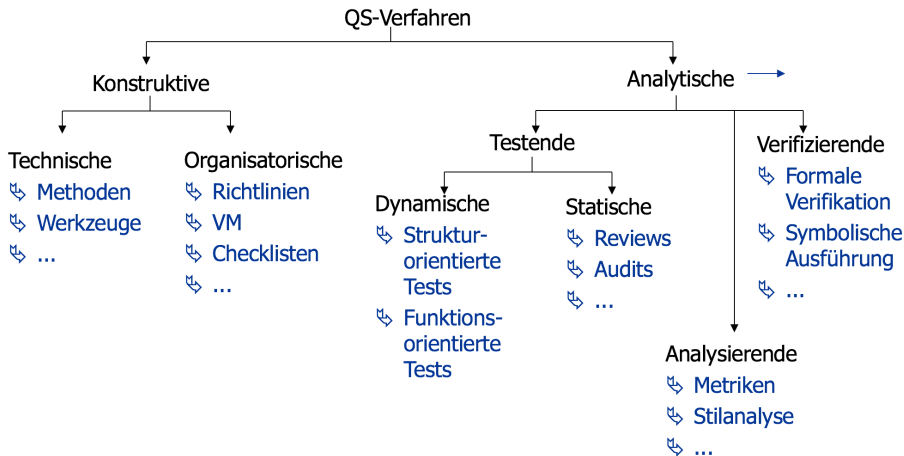
1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

Wäre es nicht super, wenn man Coding Standards und Metriken
automatisiert prüfen könnte?

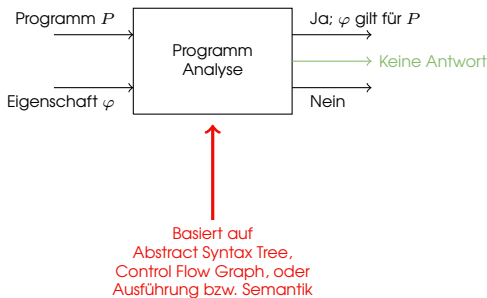
Agenda

1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

QS - Klassifizierung von Ansätzen



Programmanalyse



Beispiele:

- Alle **Klassennamen** beginnen mit Großbuchstaben
- Alle **Variablen** werden initialisiert
- **Anweisung** L10 kann nicht erreicht werden

Wdh. Getypte Ausdrücke

$a \in \mathbf{AExp}$ (Arithmetische Ausdrücke)

$a ::= x \mid \mathbf{n} \mid (a_1 \text{ } op_a \text{ } a_2)$

$b \in \mathbf{BExp}$ (Boolesche Ausdrücke)

$b ::= y \mid \mathbf{true} \mid \mathbf{false} \mid \neg b \mid (b_1 \text{ } op_b \text{ } b_2) \mid (a_1 \text{ } op_r \text{ } a_2)$

x Variable Typ $\mathbf{Int}$

y Variable Typ $\mathbf{Boolean}$

$\mathbf{n}$ Literal für $n \in \mathbb{Z}$

$op_a \in \mathbf{Op}_a$ arithmetische Op.: $+$, $-$

$op_b \in \mathbf{Op}_b$ Boolesche Op.: $\wedge$, $\vee$

$op_r \in \mathbf{Op}_r$ Relationale Op.: $>$, $<$, $=$

Die While Sprache

$$\begin{aligned} S ::= & [x := a]^l \\ & | [\text{skip}]^l \\ & | [S_1; S_2]^l \\ & | \text{if } [b]^l \text{ then } S_1 \text{ else } S_2 \\ & | \text{while } [b]^l \text{ do } S \end{aligned}$$

x Variable Typ `Int` oder `Boolean`

a Ausdruck von passendem Typ zu x

b Ausdruck aus **BExp**

$l \in \mathbf{Lab}$ Label

Stmt : Menge aller syntaktisch korrekten While Programme.

Annahme: Label identifizieren Anweisungen eindeutig!

Beispiel: Syntax

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

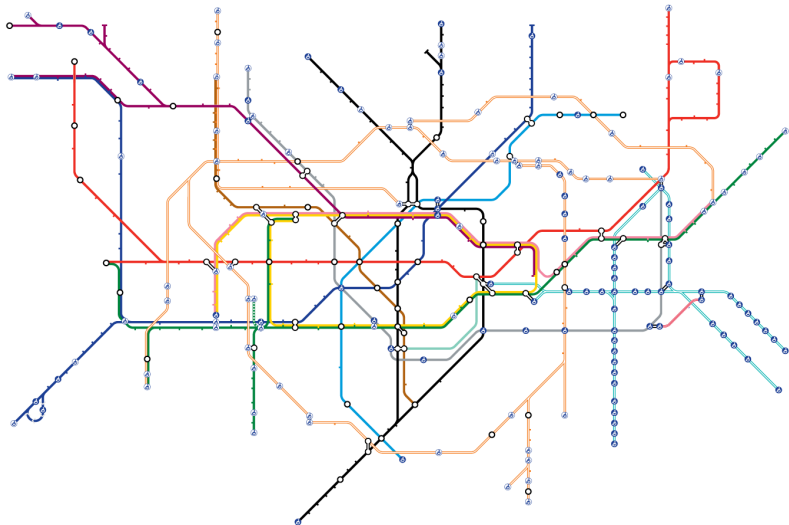
Später:

- Def. von Variablen
- Ausführung

Agenda

1. Motivation
2. Begriffe und Definitionen
- 3. Statische Code Analyse**
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

Statische Analyse

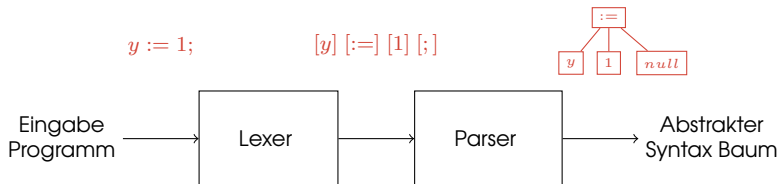


Statische Analyse

- Analyse beim Kompilieren
- Basiert auf Modell, AST oder CFG
- Prüft Eigenschaften ohne Code auszuführen
- Nutzt oft **Über-Approximierung** von möglichem Verhalten des Codes (kein Beispiel in dieser Vorlesung!)

Parsen / Übersetzen von While

Compiler Front End:



Abstrakter Syntax Baum (AST)

Intuitiv:

Ein abstrakter Syntax Baum ist ein Baum, der die **syntaktische Struktur** eines Programms ausdrückt

- Ausgabe eines Compiler Front-end
- Grundlage für für Generierung von Code



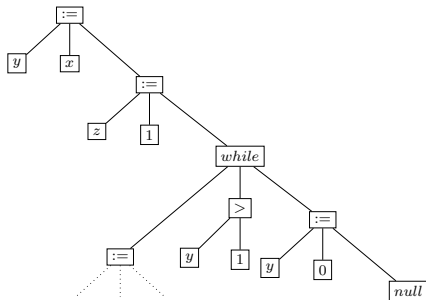
- Dient auch als Grundlage für einfache statische Code-Analyse.
- Z.B.: **Prüfe, dass alle **Namen von Klassen** mit Großbuchstaben beginnen**

AST Beispiel

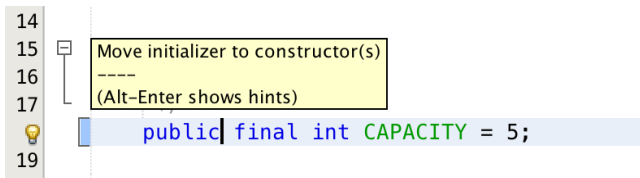
While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

Abstrakter Syntax Baum:



AST-basierte Analyse



AST-basierte Analyse

Definition (AST-basierte statische Code-Analyse)

AST-basierte statische Code-Analyse sucht nach Merkmalen / Sttruktur im abstrakten Syntax Baum eines Programms.

Beispiele:

- Initialisierung von Variablen
- Benennung von Klassen
- ...

AST-basierte Analyse: Beispiel

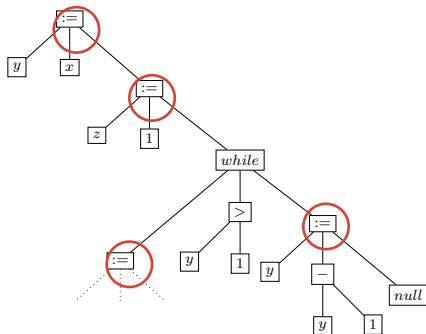
Abstrakter Syntax Baum:

While Programm:

```

 $[y := x]^1;$ 
 $[z := 1]^2;$ 
while  $[y > 1]^3$  do
   $[z := x * y]^4;$ 
   $[y := y - 1]^5$ 
 $[y := 0]^6;$ 

```

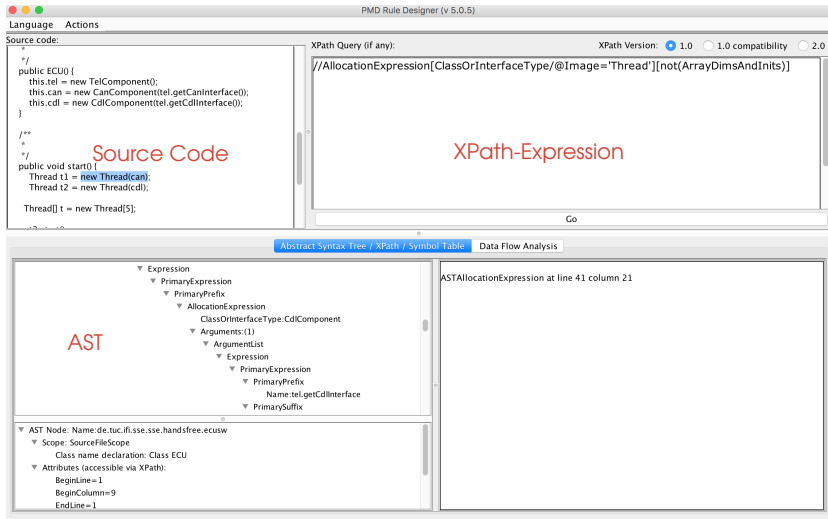


Länge von Variablennamen:

(:=)-Knoten, an denen erstes Kind ein zu kurzes Label (z.B. < 3) hat

- Bibliothek für statische Analyse
- Viele unterstützte Sprachen: C, MatLab, Java, ...
- IDE Integration
- Maven Integration

PMD Rule Designer



Natürlich funktioniert das auch für Modelle!

Kontrollflussgraph (CFG)

Intuitiv:

Der Kontrollflussgraph eines Programms P ist ein Graph (V, E) mit

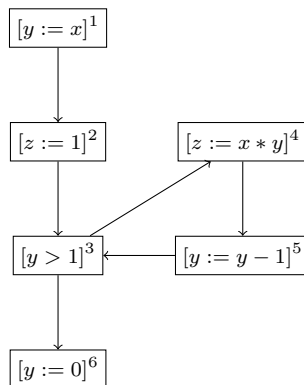
- Menge Knoten V , die einen Knoten für jede Anweisung in P enthält und
- Menge Kanten $E \subseteq V \times V$, die eine Kante $(v, v') \in E$ enthält wenn P von v zu v' übergehen kann.

CFG Beispiel

While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
    [z := x * y]4;  
    [y := y - 1]5  
[y := 0]6;
```

CFG:



Formalisierung Kontrollfluss (1)

Elementare Blöcke eines Programms: $blocks : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Blocks})$

$$blocks([x := a]^l) = \{ [x := a]^l \}$$

$$blocks([\mathbf{skip}]^l) = \{ [\mathbf{skip}]^l \}$$

$$blocks([S_1; S_2]^l) = blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{if} [b]^l \mathbf{then} S_1 \mathbf{else} S_2) = \{ [b]^l \} \cup blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{while} [b]^l \mathbf{do} S) = \{ [b]^l \} \cup blocks(S)$$

Labels eines Programms: $labels : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Lab})$

$$labels(S) = \{ l \in \mathbf{Lab} : [B]^l \in blocks(S) \}$$

Blocks: elementare Blöcke der Form $[x := a]^l$, $[\mathbf{skip}]^l$ bzw. $[b]^l$

Formalisierung Kontrollfluss (2)

Start-Knoten: $init : \text{Stmt} \rightarrow \text{Lab}$

$$init([x := a]^l) = l$$

$$init([\text{skip}]^l) = l$$

$$init([S_1; S_2]^l) = init(S_1)$$

$$init(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = l$$

$$init(\text{while } [b]^l \text{ do } S) = l$$

Formalisierung Kontrollfluss (2)

Start-Knoten: $init : \text{Stmt} \rightarrow \text{Lab}$

$$init([x := a]^l) = l$$

$$init([\text{skip}]^l) = l$$

$$init([S_1; S_2]^l) = init(S_1)$$

$$init(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = l$$

$$init(\text{while } [b]^l \text{ do } S) = l$$

End-Knoten: $final : \text{Stmt} \rightarrow \mathcal{P}(\text{Lab})$

$$final([x := a]^l) = \{l\}$$

$$final([\text{skip}]^l) = \{l\}$$

$$final([S_1; S_2]^l) = final(S_2)$$

$$final(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = final(S_1) \\ \cup final(S_2)$$

$$final(\text{while } [b]^l \text{ do } S) = \{l\}$$

Formalisierung Kontrollfluss (3)

Kontrollfluss: $flow : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Lab} \times \mathbf{Lab})$

$$flow([x := a]^l) = \emptyset$$

$$flow([\mathbf{skip}]^l) = \emptyset$$

$$flow([S_1; S_2]^l) = \{ (l, init(S_2)) : l \in final(S_1) \}$$

$$flow(\mathbf{if} [b]^l \mathbf{then} S_1 \mathbf{else} S_2) = \{ (l, init(S_1)) \} \\ \cup \{ (l, init(S_2)) \}$$

$$flow(\mathbf{while} [b]^l \mathbf{do} S) = \{ (l, init(S)) \} \\ \cup \{ (l', l) : l' \in final(S) \}$$

Umgekehrter Kontrollfluss: $flow^R : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Lab} \times \mathbf{Lab})$

$$flow^R(S) = \{ (l, l') : (l', l) \in flow(S) \}$$

Formalisierung Kontrollfluss (4)

Definition (Kontrollflussgraph (CFG))

Der Kontrollflussgraph eines *While* Programms $S_\star$ ist ein Graph (V, E) mit

- Knoten $V = labels(S_\star)$ und
- Kanten $E = flow(S_\star)$.

Annahmen: (vereinfacht Zyklomatische Komplexität)

- Isolierter Start-Knoten: $\forall l \in labels(S_\star) . (l, init(S_\star)) \notin flow(S_\star)$
- Isolierter End-Knoten: $\forall l \in labels(S_\star) . (final(S_\star), l) \notin flow(S_\star)$

(Kann durch Hinzufügen von `skip` Anweisungen erreicht werden)

Zyklomatische Komplexität (CC)

Definition (Zyklomatische Komplexität (CC))

Die Zyklomatische Komplexität eines Programms P entspricht der Anzahl der unabhängigen Pfade in P . Sei e die Zahl der Kanten im Kontrollflussgraph von P , n die Zahl der Knoten im Kontrollflussgraph und p die Zahl der Zusammenhangskomponenten^(*). Dann gilt

$$CC = e - n + 2p$$

- Komplexität wird gemessen durch Anzahl der Pfade
- Viele Pfade $\Leftrightarrow$ hohe Komplexität
- $p = 1$ für Programme mit einem Einstiegspunkt.

(*) : hat z.B. eine Bedeutung für CC eine Java Klasse

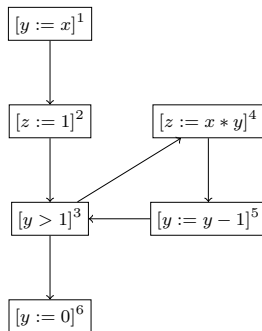
Für SWK irrelevant!

Zyklomatische Komplexität: Beispiel

While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

Kontrollflussgraph:



$$CC = 6 - 6 + 2 = 2$$

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
- 4. Strukturierte Operationelle Semantik**
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

Semantik einer Programmiersprache

- In frühen Programmiersprachen war die Semantik in natürlicher Sprache definiert
- Konsequenz: Ungenauigkeit, nicht definiertes Verhalten

Beispiel:

```
integer procedure awkward
begin
  x := x+1
  awkward := 3
end awkward
...
x := 4
z := x+awkward
```

- Globale Variable x.
- Was ist der Wert von $x + \text{awkward}$?

Formale Semantik

Idee: **Formale** Semantik für Programme, die Bedeutung (= was bei der Ausführung passiert) eindeutig festlegt.

Strukturierte Operationelle Semantik:

- Strukturiert: Feste Regeln in strukturierter Darstellung
- Operationell: Beschreibt die Ausführung
- Semantik: Bedeutung der Syntax eines Programms

Beispiel:

- While Sprache.

Formale Semantik für While

Wir benötigen:

- "Abstrakte Maschine": Formaler Begriff einer Zustandes
- Konfiguration: Kombination aus Programm und Zustand
- Ausführung: Übergänge zwischen Kombinationen

Zustand = Belegung von Variablen

Definition (Belegung (Wdh.))

Eine Belegung v von X ist eine Typ-konsistente Zuweisung von Werten zu Variablen in X . v ist eine Funktion und für $x \in X$ ist $v(x)$ der Wert mit dem x in v belegt ist.

Sei V_X die Menge aller möglichen typ-konsistenten Belegungen von X .

Update des Zustandes:

Wir schreiben: $v' = v[x \mapsto e]$ für:

$$v'(y) = \begin{cases} v(y) & \text{wenn } y \neq x, \\ e & \text{wenn } y = x. \end{cases}$$

Wobei: $x \in X$ und e konkreter Wert, der zum Typ von x passt.

Konfigurationen

Definition (Konkrete Konfiguration)

Eine **konkrete Konfiguration** ist ein Tupel $\langle S, v \rangle$ aus

- einem Programmtext $S \in \mathbf{Stmt}$ und
- einer Belegung $v \in V_X$.

Beispiel:

- Konfiguration $\sigma = \langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto 2\} \rangle$

Idee:

- S speichert den noch auszuführenden Teil des Programms
- v speichert den aktuellen Wert aller Variablen

Wir nehmen im folgenden immer an, dass $X = \text{Vars}(S_*)$
wenn wir das While Programm S_* analysieren.

Auswertung von Ausdrücken

- Set von Variablen X
- Ausdrücke aus $\mathbf{AExp} \cup \mathbf{BExp}$ über Variablen aus X

Auswertung:

- Funktion $\alpha_X : \mathbf{AExp} \times V_X \mapsto \mathbb{Z}$
- Funktion $\beta_X : \mathbf{BExp} \times V_X \mapsto \{0, 1\}$

Beispiel (Negation):

- $\beta_X[(\neg b), v] = 1$ gdw. $\beta_X[b, v] = 0$ (sonst 0)

Übergangsregeln

Wir definieren einige Regeln für das Abarbeiten von Anweisungen

Inferenzregeln:

Format: $\frac{\text{Prämissen}}{\text{Konklusion}}$

Beispiel: $\frac{a \Rightarrow b, b \Rightarrow c}{a \Rightarrow c}$

Axiome:

Format: $\overline{\text{Axiom}}$ (oder einfach: Axiom)

Beispiel: $n + 0 = n$

- Generell haben Prämisse, Konklusion und Axiome die Form $A \Rightarrow B$.
- Bedeutung: A kann in B überführt werden.
- Dabei sind A und B Konfigurationen oder Zustände

SOS: Skip

Das Axiom SKIP gibt an, wie eine `skip` Anweisung ausgeführt wird:

$$\langle \text{skip}, v \rangle \Rightarrow v \quad [\text{SKIP}]$$

- Die Konfiguration $\langle \text{skip}, v \rangle$ wird in den Zustand v überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle \text{skip}, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 2\} \quad [\text{SKIP}]$$

SOS: Zuweisung

Das Axiom ASS gibt an, wie eine Zuweisung $x := a$ ausgeführt wird:

$$\langle x := a, v \rangle \Rightarrow v[x \mapsto c] \quad \text{für } c = \alpha_X[a, v] \quad [\text{ASS}]$$

- Die Konfiguration $\langle x := a, v \rangle$ wird in den Zustand $v[x \mapsto c]$ überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle x := 5 + x, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 7\} \quad [\text{ASS}]$$

SOS: Kontrollstrukturen

Die Axiome IF_T und IF_F geben an, wie eine `if then else` Anweisung ausgeführt wird:

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_1, v \rangle \quad \text{falls } \beta_X[b, v] = 1 \quad [\text{IF}_T]$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_2, v \rangle \quad \text{falls } \beta_X[b, v] = 0 \quad [\text{IF}_F]$$

- Die Konfiguration $\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle$ wird in die Konfiguration $\langle S_1, v \rangle$ überführt falls in s die Bedingung b gilt; falls b nicht gilt wird nach $\langle S_2, v \rangle$ überführt
- Die Regeln überführen eine Konfiguration in eine Konfiguration
- Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip}, \{x \mapsto 2\} \rangle \Rightarrow \langle x := 0, \{x \mapsto 2\} \rangle \quad [\text{IF}_T]$$

SOS: Schleifen

Die Axiome WH_T und WH_F geben an, wie eine Schleife ausgeführt wird. Die Schleife wird durch die Regeln einmal "abgerollt":

$$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow \langle S; \text{while } b \text{ do } S, v \rangle \quad \text{falls } \beta_X[b, v] = 1 \quad [\text{WH}_T]$$

$$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow v \quad \text{falls } \beta_X[b, v] = 0 \quad [\text{WH}_F]$$

- Die Regel WH_T überführt eine Konfiguration in eine Konfiguration. Nach der Überführung verbleibt etwas auszuführen.
- Die Regel WH_F überführt eine Konfiguration in einen Zustand. Nach der Überführung verbleibt nichts auszuführen.

Beispiel:

$$\begin{aligned} &\langle \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto 2\} \rangle \Rightarrow \\ &\langle x := x - 1; \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto 2\} \rangle \quad [\text{WH}_T] \end{aligned}$$

SOS: Komposition

Inferenzregeln für Komposition geben an, wie Sequenzen von Anweisungen abzuarbeiten sind. Diese Regeln werden immer mit einem Axiom kombiniert:

$$\frac{\langle S_1, v \rangle \Rightarrow \langle S'_1, v' \rangle}{\langle S_1; S_2, v \rangle \Rightarrow \langle S'_1; S_2, v' \rangle} \quad [\text{COMP}_1]$$

$$\frac{\langle S_1, v \rangle \Rightarrow v'}{\langle S_1; S_2, v \rangle \Rightarrow \langle S_2, v' \rangle} \quad [\text{COMP}_2]$$

- Regel COMP_1 funktioniert mit Regeln IF_T , IF_F und WH_T
- Regel COMP_2 funktioniert mit ASS und SKIP und WH_F
- Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\frac{\langle x := 1, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 1\}}{\langle x := 1; \text{skip}, \{x \mapsto 2\} \rangle \Rightarrow \langle \text{skip}, \{x \mapsto 1\} \rangle} \quad [\text{COMP}_2 + \text{ASS}]$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Weitere Beispiele auf Papier ...

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Weitere Beispiele auf Papier ...

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

...

[COMP₂ + WH_F] ($y \leq 1$) :

$$\frac{\langle S_3, v_4 \rangle \Rightarrow v_4}{\langle S_3; S_6, v_4 \rangle \Rightarrow \langle S_6, v_4 \rangle}$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

...

[COMP₂ + WH_F] ($y \leq 1$) :

$$\frac{\langle S_3, v_4 \rangle \Rightarrow v_4}{\langle S_3; S_6, v_4 \rangle \Rightarrow \langle S_6, v_4 \rangle}$$

[ASS] :

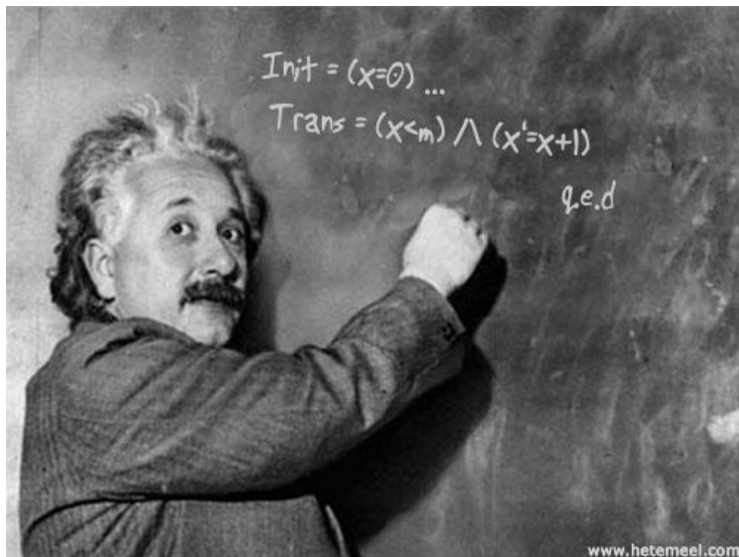
$$\langle y := 0, v_4 \rangle \Rightarrow v_4[y \mapsto 0]$$

Weitere Beispiele auf Papier ...

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

Symbolische Analyse



Symbolische Ausführung

Definition (Symbolische Ausführung)

Symbolische Ausführung ist eine statische Analyse, bei der ein Programm mit symbolischen Werten anstelle von konkreten Werten ausgeführt wird.

- Alle Anweisungen werden symbolisch berechnet.
- Bei der Ausführung von Kontrollstrukturen können ggf. beide Zweige ausgeführt werden.
- Die auf einem Pfad der Ausführung entstehenden Einschränkungen von Variablen werden als Pfadbedingungen aufgezeichnet.

Beispiel:

Concrete: $i = 1$

```
int y = i + 100; // y = 101
if (y > 200) {
    ...
} else {
    ... // here
}
```

Symbolic: $i = X$

```
int y = i + 100; // y = X+100
if (y > 200) {
    ... // X+100 > 200
} else {
    ... // X+100 <= 200
}
```

SE: Konfigurationen

Symbolische Belegung:

- Menge symbolischer Variablen $\mathbf{Sym} = \{A, B, \dots\}$
- Symbolische Belegung $\mathfrak{v} : X \mapsto \mathbb{F}_{\mathbf{Sym}}$
- $\mathfrak{V}_X$: Menge aller symbolischen Belegungen von X

Symbolisches Auswerten:

Für $X = \{x_1, \dots, x_k\}$ und $a \in \mathbb{F}_X$ ist $\sigma_X : \mathbb{F}_X \times \mathfrak{V} \mapsto \mathbb{F}_{\mathbf{Sym}}$ so dass:

$$\sigma_X[a, \mathfrak{v}] =_{def} a[\mathfrak{v}(x_1)/x_1][\dots][\mathfrak{v}(x_k)/x_k] \in \mathbb{F}_{\mathbf{Sym}}$$

Beispiel:

- Sei $\mathfrak{v} = \{x_1 \mapsto (A + 2)\}$ und $a = 'x_1 > 0'$
- Dann ist $\sigma_X[a, \mathfrak{v}] = '(A + 2) > 0'$

$\mathbb{F}_{\mathbf{Sym}}$: Ausdrücke aus **Exp** mit Var. aus **Sym**

SE: Skip

Das Axiom SKIP^{Sym} gibt an, wie eine `skip` Anweisung symbolisch ausgeführt wird:

$$\langle \text{skip}, \mathbf{v}, \varphi \rangle \Rightarrow (\mathbf{v}, \varphi) \quad [\text{SKIP}^{\text{Sym}}]$$

- Die Konfiguration $\langle \text{skip}, \mathbf{v}, \varphi \rangle$ wird in den symbolischen Zustand $(\mathbf{v}, \varphi)$ überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle \text{skip}, \{x \mapsto (A + 2)\}, (A > 10) \rangle \Rightarrow (\{x \mapsto (A + 2)\}, (A > 10)) \quad [\text{SKIP}^{\text{Sym}}]$$

SE: Zuweisung

Das Axiom ASS^{Sym} gibt an, wie eine Zuweisung $x := a$ symbolisch ausgeführt wird:

$$\langle x := a, \mathbf{v}, \varphi \rangle \Rightarrow (\mathbf{v}[x \mapsto t], \varphi) \quad \text{für } t = \sigma_X[a, \mathbf{v}] \quad [\text{ASS}^{\text{Sym}}]$$

- Die Konfiguration $\langle x := a, \mathbf{v}, \varphi \rangle$ wird in den Zustand $(\mathbf{v}[x \mapsto t], \varphi)$ überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle x := 5 + x, \{x \mapsto (A + 2)\}, \text{true} \rangle \Rightarrow (\{x \mapsto 5 + (A + 2)\}, \text{true}) \quad [\text{ASS}^{\text{Sym}}]$$

SE: Kontrollstrukturen

Die Axiome IF_T^{Sym} und IF_F^{Sym} geben an, wie eine `if then else` Anweisung symbolisch ausgeführt wird:

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v, \varphi \rangle \Rightarrow \langle S_1, v, (\varphi \wedge \sigma_X[b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[b, v] \text{ sat.} \quad [\text{IF}_T^{\text{Sym}}]$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v, \varphi \rangle \Rightarrow \langle S_2, v, (\varphi \wedge \sigma_X[\neg b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[\neg b, v] \text{ sat.} \quad [\text{IF}_F^{\text{Sym}}]$$

- Beachte: Es können oft beide Regeln angewendet werden.
Es entsteht ein symbolischer Ausführungsbaum!
- Die Regeln überführen eine symbolische Konfiguration in eine symbolische Konfiguration. Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle x := 0, \{x \mapsto A\}, (A > 0) \rangle \quad [\text{IF}_T^{\text{Sym}}]$$

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle \text{skip, } \{x \mapsto A\}, \neg(A > 0) \rangle \quad [\text{IF}_F^{\text{Sym}}]$$

Symbolische Pfadbedingung $\varphi \in \mathbb{F}_{\text{Sym}}$

SE: Schleifen

Die Axiome WH_T^{Sym} und WH_F^{Sym} geben an, wie eine Schleife symbolisch ausgeführt wird. Die Schleife wird durch die Regeln einmal "abgerollt":

$$\langle \text{while } b \text{ do } S, v, \varphi \rangle \Rightarrow \langle S; \text{while } b \text{ do } S, v, (\varphi \wedge \sigma_X[b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[b, v] \text{ sat.} \quad [\text{WH}_T^{\text{Sym}}]$$

$$\langle \text{while } b \text{ do } S, v, \varphi \rangle \Rightarrow (v, (\varphi \wedge \sigma_X[\neg b, v])) \quad \text{falls } \varphi \wedge \sigma_X[\neg b, v] \text{ sat.} \quad [\text{WH}_F^{\text{Sym}}]$$

- Die Regel WH_T^{Sym} überführt eine symbolische Konfiguration in eine symbolische Konfiguration. Nach der Überführung verbleibt etwas auszuführen.
- Die Regel WH_F^{Sym} überführt eine symbolische Konfiguration in einen symbolischen Zustand. Nach der Überführung verbleibt nichts auszuführen.

Beispiel:

$$\begin{aligned} &\langle \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto A\}, (A > 1) \rangle \Rightarrow \\ &\langle x := x - 1; \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto A\}, (A > 1 \wedge A > 0) \rangle \quad [\text{WH}_T^{\text{Sym}}] \end{aligned}$$

Regel WH_F^{Sym} im Beispiel nicht anwendbar, da $((A > 1) \wedge \neg(A > 0))$ nicht erfüllbar!

SE: Komposition

Inferenzregeln für Komposition geben an, wie Sequenzen von Anweisungen abzuarbeiten sind. Diese Regeln werden immer mit einem Axiom kombiniert:

$$\frac{\langle S_1, \mathbf{v}, \varphi \rangle \Rightarrow \langle S'_1, \mathbf{v}', \varphi' \rangle}{\langle S_1; S_2, \mathbf{v}, \varphi \rangle \Rightarrow \langle S'_1; S_2, \mathbf{v}', \varphi' \rangle} \quad [\text{COMP}_1^{\text{Sym}}]$$

$$\frac{\langle S_1, \mathbf{v}, \varphi \rangle \Rightarrow (\mathbf{v}', \varphi')}{\langle S_1; S_2, \mathbf{v}, \varphi \rangle \Rightarrow \langle S_2, \mathbf{v}', \varphi' \rangle} \quad [\text{COMP}_2^{\text{Sym}}]$$

- Regel $\text{COMP}_1^{\text{Sym}}$ funktioniert mit Regeln IF_T^{Sym} , IF_F^{Sym} und WH_T^{Sym}
- Regel $\text{COMP}_2^{\text{Sym}}$ funktioniert mit ASS^{Sym} und SKIP^{Sym} und WH_F^{Sym}

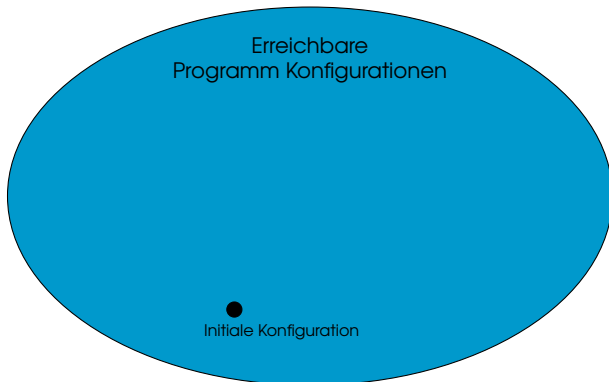
Beispiel:

$$\frac{\langle x := 1, \{x \mapsto A\}, \text{true} \rangle \Rightarrow (\{x \mapsto 1\}, \text{true})}{\langle x := 1; \text{skip}, \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle \text{skip}, \{x \mapsto 1\}, \text{true} \rangle} \quad [\text{COMP}_2^{\text{Sym}} + \text{ASS}^{\text{Sym}}]$$

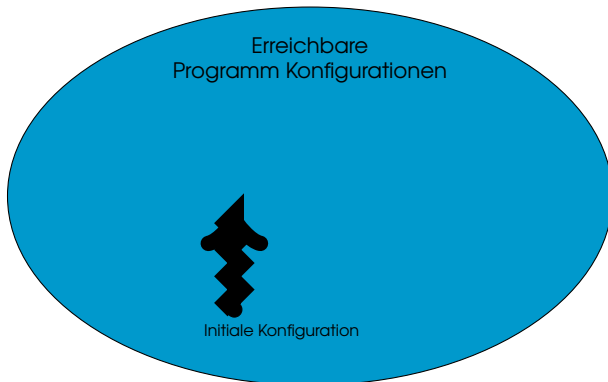
Beispiel

Auf Papier

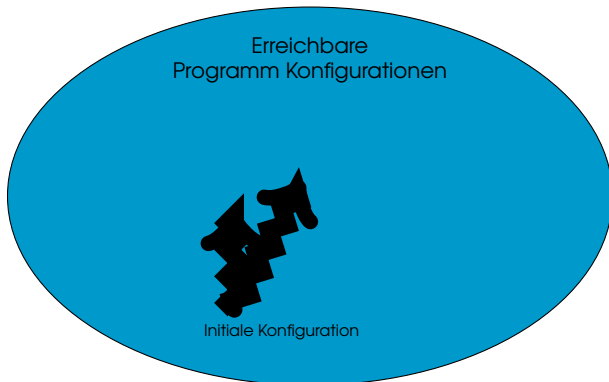
Symbolische Ausführung: Intuition



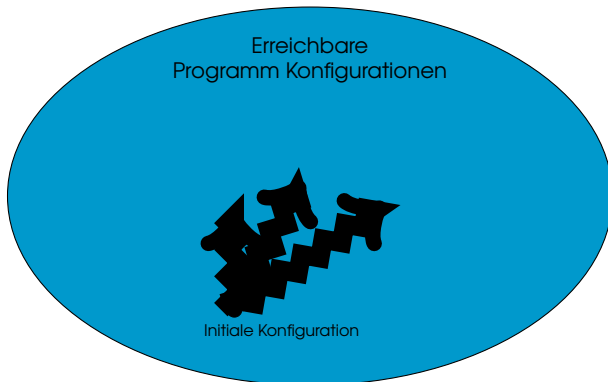
Symbolische Ausführung: Intuition



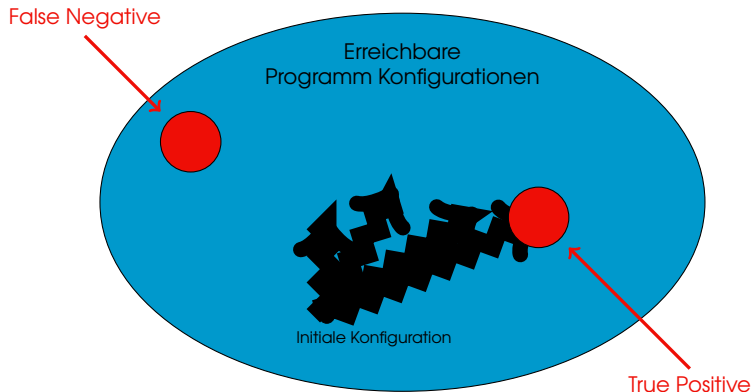
Symbolische Ausführung: Intuition



Symbolische Ausführung: Intuition



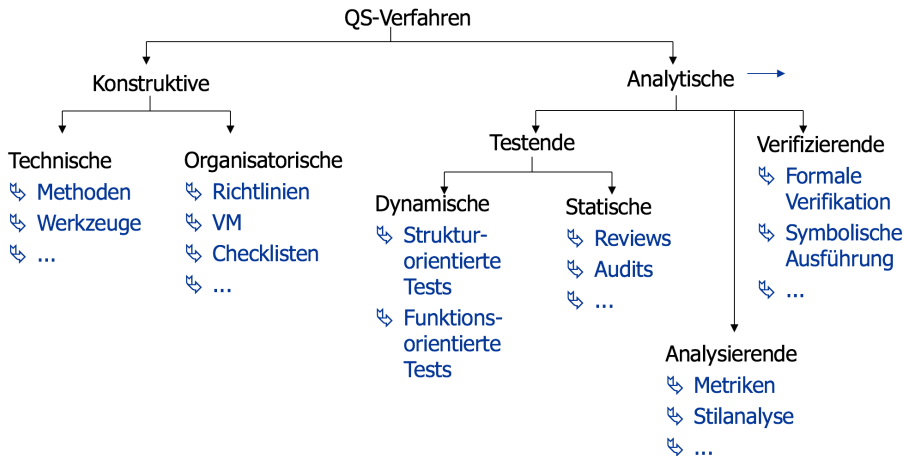
Symbolische Ausführung: Intuition



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

Zusammenfassung



Zusammenfassung Programmanalyse

AST-basiert:

- Suche nach Mustern im abstrakten Syntax Baum
- Einfache Eigenschaften (z.B. Benennung)

CFG-basiert:

- Komplexere Code Metriken (z.B. CC)
- Datenfluss Analyse
- Über-Approximierung von Datenfluss

Ausführungssemantik:

- z.B. Symbolische Ausführung auf Basis von SOS
- Hoare Beweise
- Exakte Modellierung der Ausführungssemantik

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Statische Code Analyse
4. Strukturierte Operationelle Semantik
5. Symbolische Ausführung
6. Zusammenfassung
7. Standards, Literatur und Referenzen

- Liggesmeyer: Software-Qualität Testen, Analysieren und Verifizieren von Software; Springer Spektrum, 2009.
- Nielson, Nielson, Hankin: Principles of Program Analysis; Springer 1999.
- King: Symbolic Execution and Program Testing; Communications of the ACM (CACM), 19 (7), 1976.